



Trabalhe no Bruxó

P6 - Apresentação Final

Rafaela Grillo & Theo Albuquerque & Yuri Sacksida

Espectativas vs. Realidade



Espectativas

- ☐ mecânicas básicas feitas:
 - ☐ mesclagem de cartas
 - ☐ diálogos visualizados com texto
 - ☐ diálogos visualizados com personagem
 - ☐ escolhas tomáveis com respostas nos diálogos
 - ☐ escolhas tomáveis com criação de poções nos diálogos
- ☐ conteúdo básico feito:
 - ☐ diálogo de 1 caso
 - ☐ artes
 - ☐ de cartas suficientes pras opções do primeiro caso
 - ☐ do fundo pra mesa
 - ☐ do fundo pra loja
 - ☐ dos personagens do caso
- ☐ extras:
 - ☐ animações ao arrastar as cartas
 - ☐ partículas ou efeito ao mesclar as cartas
 - ☐ “levantar” cartas ao segurar
 - ☐ expressões de personagens

Realidade

- ☐ (90%) mecânicas básicas feitas:
 - ☒ mesclagem de cartas
 - ☒ diálogos visualizados com texto
 - ☐ diálogos visualizados com personagem
 - ☒ escolhas tomáveis com respostas nos diálogos
 - ☒ escolhas tomáveis com criação de poções nos diálogos
- ☐ (55%) conteúdo básico feito:
 - ☐ (60%) diálogo de 1 caso
 - ☐ artes
 - ☒ de cartas suficientes pras opções do primeiro caso
 - ☐ (só tem o esboço) do fundo pra mesa
 - ☐ (só tem o esboço) do fundo pra loja
 - ☐ dos personagens do caso
- ☐ extras:
 - ☒ (meio gambiarrada) linguagem e interpretador pros diálogos
 - ☐ animações ao arrastar as cartas
 - ☐ partículas ou efeito ao mesclar as cartas
 - ☐ “levantar” cartas ao segurar
 - ☐ expressões de personagens

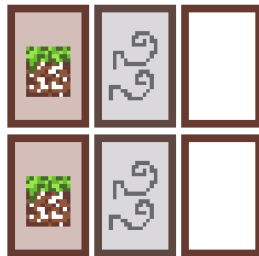
Estado do jogo



Elementos



Elementos



Elementos



Diálogos

```
theo@komputilo: ~/src/trabalhe-no-bruxo/docs
77 [
1 [Medeia] Mais alguma coisa?
2 [Pedro] Você tem uma saia?
3 [Medeia] Se vira! Isso aqui não é um brechó, não!
4 [Pedro] Não é?
5
6 [Medeia]
7 - Espera, na verdade é sim! Vai lá na frente que a gente tá vendendo..
8 - Mas vai te custar separado!
9 > Pedro sai, deixando Medeia para pensar na receita.
10
11 [Medeia]
12 - (Alguma coisa que pode moldar o corpo dele à sua imaginação? Eu não acho que tenho algo par
a isso...)
13 - (... mas que coisa abstrata esse Pedro foi inventar também! Mas não tem jeito, esse é o tra
balho que eu sempre quis então eu tenho que me virar!)
14 > [MESA:pocao_bom](final_bom)
15 > [MESA:pocao_som](final_ruim)
16
17 # final_bom
18
19 [Medeia]
20 - (Ótimo! Usando os poderes da argila vai ser muito fácil modelar o próprio corpo. Mas eu ain
da preciso dar um jeito de sintetizar isso em uma poção...)
21 > Medeia mistura Argila com vazio
22 - (Consegui! Eu decifrei minha primeira receita!!)
23
24 - [nada mais feito ainda]
25
26 [](#lm)
27
28 # final_ruim
29
30 [Medeia]
31 - (Som?! Bem... Pelo menos isso vai melhorar aquela voz insuportável dele...)
32 > Medeia volta ao balcão
33 - (Err... Eu tenho quase certeza que isso vai funcionar! Qual a pior coisa que poderia aconte
cer?)
34 [Pedro]
35 - É aí? A parada tá pronta?
36
```

../assets/diálogo.mvp.md 77,0-1 25%

Diálogos

```
theo@komputilo: ~/src/trabalhe-no-bruxo/docs
17 [Pedro]
16 - ... Eu me apaixonei.
15 [Medeia]
14 - (Me parece que você devia parar de chamar essas coisas de idiotice.)
13 - E o que a gente tem a ver com isso?
12 [Pedro]
11 - Eu... falei pra ele que era uma menina.
10
9 > Ele mostra a descrição de seu, está escrito: "totalmente uma garota, pode confiar rsrs"
8
7 - Eu preciso de alguma coisa que me faça parecer uma menina pra ele.
6
5 [Medeia]
4 - (E lá está. Agora sim é o Pedro que eu conheço.)
3 > [Você já pensou em só pagar por uma cirurgia que nem os outros?](pedro_1)
2 > [Sinto muito, não posso te ajudar com isso.](pedro_2)
1 > [Tudo bem! Aceitamos o seu caso.](pedro_3)
58
1 # pedro_1
2 [Pedro] Tá maluca?? Não nasci herdeiro, não!
3
4 [](pedro_3)
5
6 # pedro_2
7 [Pedro] Não foi isso que a placa de vocês disse! Pô Medeia, me ajuda aí.
8
9 [](pedro_3)
10
11 # pedro_3
12 [Medeia]
13 - Ok. Me descreve exatamente as coisas que você precisa que mudem.
14 [Pedro]
15 - Tá. Pra falar a verdade, eu acho que eu preciso de uma coisa bastante simples.
16 - Eu preciso de alguma coisa que me deixe moldar o meu corpo.
17 - Algo que aproxime ele do que eu imagino.
18 - ... e quem sabe uma dose de coragem?
19
20 [Medeia] Mais alguma coisa?
21 [Pedro] Você tem uma saia?
22 [Medeia] Se viral! Isso aqui não é um brechó, não!
```

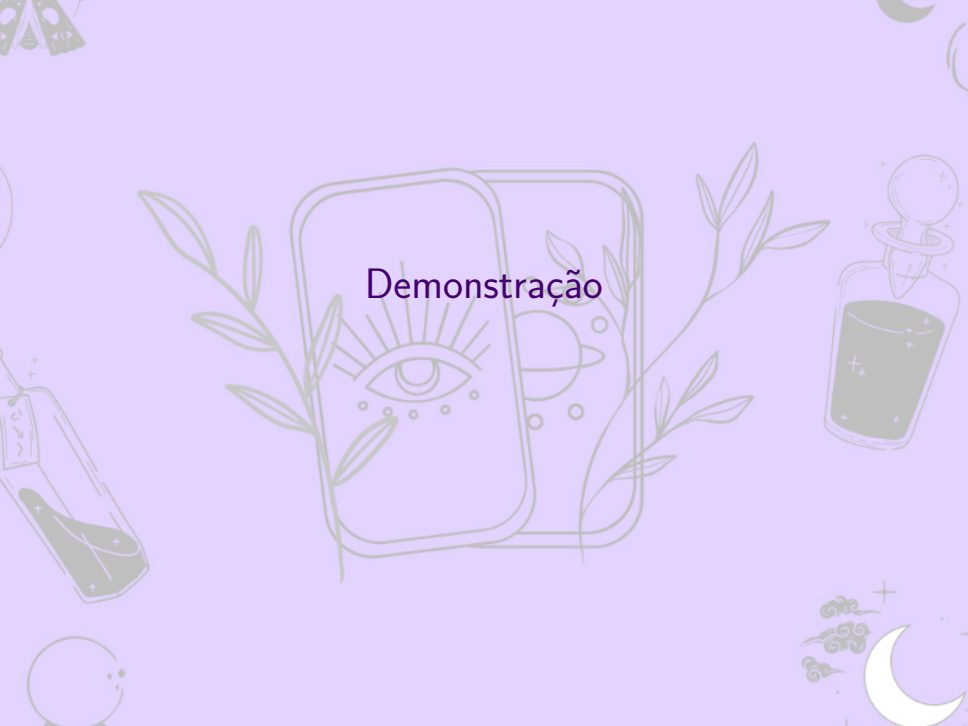
./assets/diálogo.mvp.md 58,0-1 13%

Arte

Cada carta é feita com 2 sprites, um que é um fundo igual para todas as cartas e outro que é semi-transparente com a arte específica dos elementos.



Demonstração



Específicos



Geral

O programa principal é uma máquina de estados de “telas”.

Cada tela precisa fornecer 3 funções:

- void [tela]_setup(renderer*)
- tela [tela]_loop(renderer*, event)
- void [tela]_free(void)

No momento temos 3 telas:

- MENU
- MESA
- LOJA

Geral

```
int main() { /* ... */
    enum tela tela = ZERO;
    uint32_t falta = TIMEOUT;
    for (SDL_Event evt; evt.type != SDL_QUIT; ) {
        AUX_NextEvent(&evt, &falta, TIMEOUT);

        enum tela prox;
        switch (tela) {
            case ZERO: prox = MENU; break;
            case MENU: prox = menu_loop(ren, evt); break;
            case MESA: prox = mesa_loop(ren, evt); break;
            case LOJA: prox = loja_loop(ren, evt); break;
        }

        /* ... */
        tela = prox;
    }
    /* ... */
}
```


Geral

```
bool AUX_WaitEventTimeoutCount(SDL_Event* evt, uint32_t* ms) {  
    static uint32_t antes;  
    if (antes == 0) antes = SDL_GetTicks();  
  
    bool evento = SDL_WaitEventTimeout(evt, *ms);  
    if (evento) {  
        uint32_t delta = DT(antes, &antes);  
        *ms = (delta < *ms) ? *ms - delta : 0;  
    }  
    return evento;  
}  
  
bool AUX_WaitEventTimeout(SDL_Event* evt, uint32_t* ms,  
                           uint32_t timeout) {  
    bool evento = AUX_WaitEventTimeoutCount(evt, ms);  
    if (!evento) *ms = timeout;  
  
    return evento;  
}
```

Geral

```
typedef enum {  
    AUX_TIMEOUTEVENT = 0,  
    AUX_SURECLICKEVENT,  
    AUX_FIRSTUSEREVENT,  
} AUX_EventType;  
  
void AUX_FillTimeout(SDL_Event* evt) {  
    evt->user = (SDL_UserEvent) {  
        .type = SDL_USEREVENT,  
        .code = AUX_TIMEOUTEVENT,  
        .timestamp = SDL_GetTicks(),  
    };  
}  
  
void AUX_NextEvent(SDL_Event* evt, uint32_t* falta,  
                  uint32_t timeout) {  
    if (AUX_WaitEventTimeout(evt, falta, timeout));  
    else AUX_FillTimeout(evt);  
}
```

Drag&Drop

```
typedef enum drag_drop_state {  
    UNCLICKED = 0,  
    CLICKING,  
    DRAGGING,  
  
    DRAG_STATE_COUNT,  
} DragDropState;  
  
typedef struct drag_drop_rect {  
    SDL_Rect r;  
  
    DragDropState state;  
    SDL_MouseButtonEvent click;  
    SDL_Point offset;  
} DragDropRect;
```

Drag&Drop

```
#define asClick(evt) transmute(SDL_MouseButtonEvent, evt)
void AUX_DragDropCancel(DragDropRect* self, SDL_Event evt) {
    const bool clicked = (self->state == CLICKING) || (self->state == DRAGGING);
    switch (evt.type) {
        case SDL_KEYDOWN: switch (evt.key.keysym.sym) {
            case SDLK_ESCAPE: if (clicked) {
                self->r.x = self->click.x + self->offset.x;
                self->r.y = self->click.y + self->offset.y;

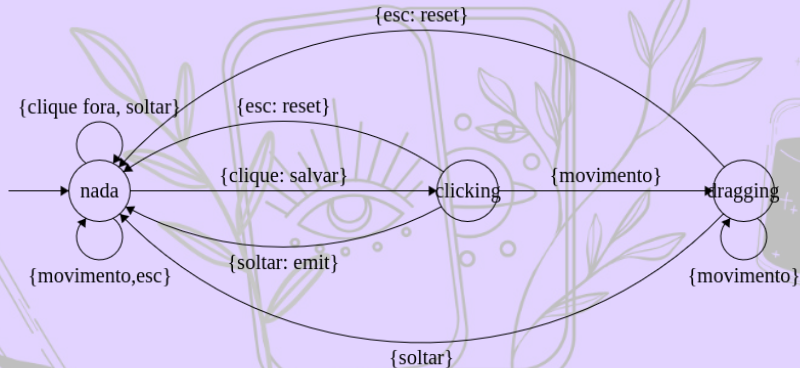
                self->state = UNCLICKED;
            } break;
        } break;
        case SDL_MOUSEBUTTONDOWN: {
            SDL_Point loc = { evt.button.x, evt.button.y };
            if (SDL_PointInRect(&loc, &self->r)) {
                self->offset.x = self->r.x - loc.x;
                self->offset.y = self->r.y - loc.y;
                self->click = asClick(evt);

                self->state = CLICKING;
            }
        } break;
        case SDL_MOUSEBUTTONUP: {
            if (self->state == CLICKING) AUX__EmitSureClick(self);

            self->state = UNCLICKED;
        } break;
        case SDL_MOUSEMOTION: if (clicked) {
            self->r.x = evt.button.x + self->offset.x;
            self->r.y = evt.button.y + self->offset.y;

            self->state = DRAGGING;
        } break;
    }
}
```

Drag&Drop



Geral

```
#define trans(prev, curr) \  
    (((uint16_t)prev<<8) | ((uint16_t)curr))  
  
#define prox_diff(prev, curr) \  
    (((prev) != (curr)) ? curr : ZERO)  
#define curr_diff(prev, curr) \  
    (((prev) != (curr)) ? prev : ZERO)  
  
#define maior(prev, curr) (prev>curr ? prev : curr)  
#define menor(prev, curr) (prev<curr ? prev : curr)  
#define par(prev, curr) \  
    trans(menor(prev, curr), maior(prev, curr))
```

Geral

```
enum tipo_carta combinar(const enum tipo_carta t1,  
                        const enum tipo_carta t2) {  
    switch (par(t1, t2)) {  
        case par(CARTA_FOGO, CARTA_AGUA): return CARTA_VAPOR;  
        case par(CARTA_TERRA, CARTA_AGUA): return CARTA_LAMA;  
        case par(CARTA_FOGO, CARTA_AR):   return CARTA_SOM;  
  
        case par(CARTA_LAMA, CARTA_NADA): return POCAO_LAMA;  
        case par(CARTA_SOM,  CARTA_NADA): return POCAO_SOM;  
  
        default: return CARTA_NADA;  
    }  
}
```

Mais eventos de usuário

```
void FillMergeEvent(SDL_Event* evt, struct carta* carta) {  
    evt->user = (SDL_UserEvent) {  
        .type = SDL_USEREVENT,  
        .code = AUX_MERGE_EVENT,  
        .data1 = (void*)carta->tipo,  
        .timestamp = SDL_GetTicks(),  
    };  
}
```

```
void EmitMergeEvent(struct carta* carta) {  
    SDL_Event evt; FillMergeEvent(&evt, carta);  
    SDL_PushEvent(&evt);  
}
```


Mesclagem de cartas

```
for (size_t i = num_cartas; i--;) {  
    AUX_DragDropCancel(&cartas[i].drag, evt);  
    if (cartas[i].drag.state != UNCLICKED) {  
        AUX_ToEndLen(cartas, num_cartas, i); break;  
    }  
}
```

Mesclagem de cartas

```
struct carta* last = &cartas[num_cartas-1];
if (last->drag.state == UNCLICKED &&
    SDL_HasIntersection(&last->drag.r, &zona_fusao)) {
    for (size_t i = num_cartas-1; i--;) {
        const struct carta* curr = &cartas[i];
        if (!SDL_HasIntersection(&last->drag.r, &curr->drag.r) ||
            !SDL_HasIntersection(&curr->drag.r, &zona_fusao))
            continue;

        struct carta n = fundir(*last, *curr);
        if (n.tipo == CARTA_NADA) continue;

        *last = n; AUX_RemoveUnordered(cartas, num_cartas, i);
        if (AUX_NullTerminatedFind(pocoes_possiveis, &n.tipo)) {
            EmitMergeEvent(&n); prox_tela = DIALOGO;
        }
        break;
    }
}
```

etc

```
static inline
#define AUX_ToEnd(arr, i) \
    AUX_ToEndLen(arr, LEN(arr), i)
#define AUX_ToEndLen(arr, len, i) \
    AUX_ToEndSzLen(arr, sizeof(*arr), len, i)
void AUX_ToEndSzLen(void* arr, size_t size, size_t len, size_t i)
    #define elem(arr, size, idx) ((arr) + (size)*(idx))
    char *const base = arr;
    char *const curr = elem(base, size, idx);

    char buf[size];
    memcpy(buf, curr, size);

    char *const next = elem(base, size, idx+1);
    char *const last = elem(base, size, len-1);

    memmove(curr, next, last-curr);
    memcpy(last, buf, size);
#undef elem
}
```

etc

```
static inline
#define AUX_Find(arr, needle) \
    AUX_FinLen(arr, LEN(arr), needle)
#define AUX_FinLen(arr, len, needle) \
    AUX_FinSzLen(arr, sizeof(*arr), len, needle)
void* AUX_FinSzLen(void* arr, size_t size, size_t len, void* ne
    #define elem(arr, size, idx) ((arr) + (size)*(idx))
    char *const base = arr;
    for (size_t i = 0; i < len; i++) {
        char *const curr = elem(base, size, i);
        if (memcmp(curr, needle, size) == 0) return curr;
    } return NULL;
#undef elem
}
```

etc

```
static inline
#define AUX_NullTerminatedLen(arr) \
    AUX_NullTerminatedLenSz(arr, sizeof(*arr))
size_t AUX_NullTerminatedLenSz(void* arr, size_t size) {
    #define elem(arr, size, idx) ((arr) + (size)*(idx))
    char zero[size]; memset(zero, 0, size);
    char *const base = arr;
    for (size_t i = 0;; i++) {
        char *const curr = elem(base, size, i);
        if (memcmp(curr, zero, size) == 0) return i;
    }
    #undef elem
}

#define AUX_NullTerminatedFind(arr, needle) \
    AUX_FinLen(arr, AUX_NullTerminatedLen(arr), needle)
```